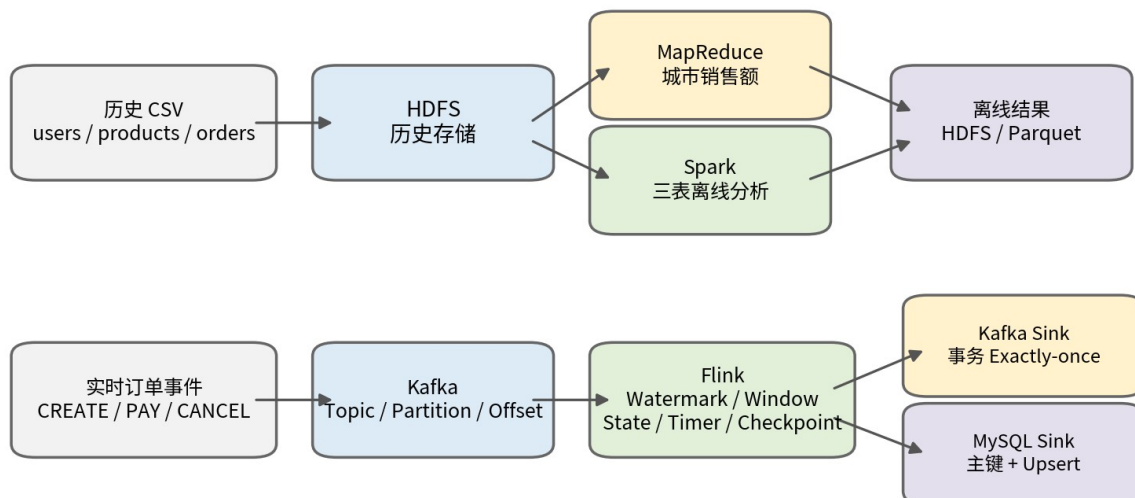


大数据综合项目实战

第 10 讲：故障注入、恢复与全系统最终验收

HDFS + MapReduce + Spark + Kafka + Flink + MySQL 最终闭环

综合项目最终系统：离线链 + 实时链



最终验收不是“某个 Job SUCCESS”，而是两条数据链的业务结果与故障恢复都可证明。

本讲定位：前 9 讲证明了“系统能跑”。第 10 讲要证明“系统坏了以后还能恢复，而且恢复后的业务结果仍然正确”。这才是综合项目真正的最后闭环。

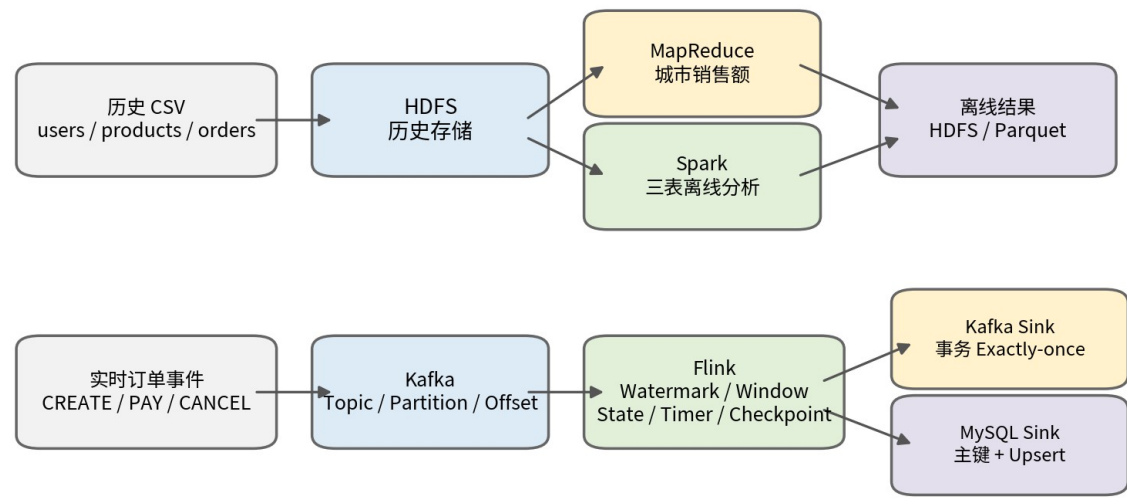
1. 本讲学习目标：从“能跑”升级到“能证明可靠”

- 能建立一个健康基线：服务、Job、Checkpoint、Kafka、MySQL 在故障前都可验证。
- 能安全地注入 TaskManager 故障，并观察 Job 恢复、State/Timer 恢复和 Kafka Source 位置恢复。
- 能解释为什么故障后会重放部分输入，但最终 managed state 仍可保持一致。
- 能注入 MySQL Sink 故障，并区分 Kafka 事务输出与 MySQL 幂等 Upsert 的不同保证。
- 能用 Savepoint 完成一次“主动停止 - 恢复”演练，理解它和自动 Checkpoint 恢复的区别。
- 能对 HDFS、MapReduce、Spark、Kafka、Flink、MySQL 做最终分层验收。
- 能形成固定排错顺序，而不是遇到异常就先重启所有服务。

最终标准：本讲不是看一个 Job 显示 RUNNING/SUCCESS，而是要求“故障恢复证据 + 外部系统结果 + 业务标准答案”三者同时成立。

2. 最终系统架构：现在你面对的是两条真正的数据链

综合项目最终系统：离线链 + 实时链



最终验收不是“某个 Job SUCCESS”，而是两条数据链的业务结果与故障恢复都可证明。

离线链回答“过去发生了什么”：历史 CSV 进入 HDFS，MapReduce 和 Spark 从同一份分布式数据读取并产出离线结果。实时链回答“现在正在发生什么”：订单事件进入 Kafka，Flink 按 Event Time 处理，维护 State/Timer/Checkpoint，再把结果写到 Kafka 和 MySQL。

因此最终验收也必须分成两部分：离线链验“可重复计算 + 结果一致”，实时链验“持续处理 + 故障恢复 + 外部交付一致性”。

3. 故障演练前的四条纪律

故障注入的四条纪律



- 1. 先做健康基线。故障前都不健康，就无法判断后面的变化到底由什么造成。
- 2. 一次只注入一个故障。不要同时停 Kafka、MySQL、TaskManager，否则你无法定位根因。

3. 每次保存证据：Flink Job 状态、Completed Checkpoint、Kafka committed 输出、MySQL 行数、业务标准答案。

4. 先恢复基础服务，再验业务结果。服务恢复只是第一层，不代表结果没有重复或丢失。

安全规则：本讲提供的危险脚本默认只 Dry Run。真正杀 TaskManager 必须显式设置 CONFIRM_KILL=YES；停 MySQL 也必须显式确认。只在教学 VM 中运行，不要在生产机器上练。

4. 每次实验统一从这里开始：启动 VM + VS Code + 环境预检

仍然按固定规范：假设虚拟机刚打开。Windows 打开 VS Code → Remote SSH 连接 Ubuntu → File / Open Folder 打开项目目录 → Terminal。

```
cd ~/bigdata-capstone
whoami
hostname
pwd

# Hadoop / Kafka / Flink / MySQL 相关环境
echo $HADOOP_HOME
echo $KAFKA_HOME
echo $FLINK_HOME
java -version
mvn -version
```

如果你已经把第 10 讲代码包解压到 `~/bigdata-capstone/lecture10-final-acceptance`，可以直接运行预检脚本：

```
cd ~/bigdata-capstone/lecture10-final-acceptance
./10_preflight.sh
```

脚本会查看 8081、9092、3306、9870、8088 等端口、JVM 进程、Flink RUNNING Job 和项目 Kafka Topic。

不要跳过：如果 9092 没监听，就先修 Kafka；如果 3306 没监听，就先修 MySQL；如果 Flink Web UI 8081 都打不开，就不要继续做故障注入。

5. 第一步：先清场，建立“健康基线”

最终验收不能在上一轮残留数据上直接继续，否则你无法判断重复行来自故障重放，还是来自上一次实验。建议先备份需要的结果，再清空第 9 讲实时结果。

```
cd ~/bigdata-capstone/lecture10-final-acceptance

# 初始化/清空 MySQL 两张实时结果表
mysql -u root -p < sql/10_init_mysql.sql

# 删除并重建 4 个教学 Topic
./10_reset_topics.sh

# 编译一次
./10_build.sh
```

然后确认：

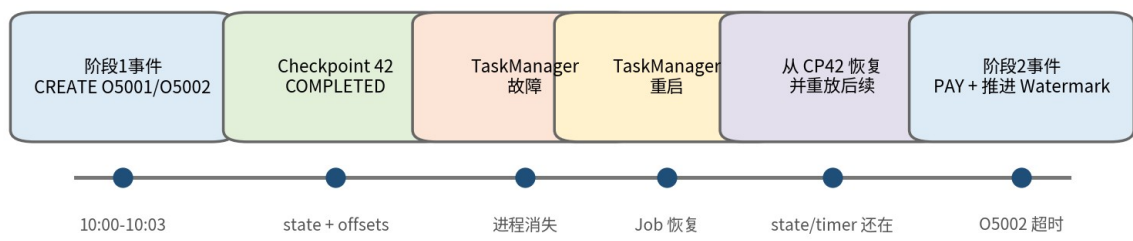
```
$FLINK_HOME/bin/flink list -r  
$KAFKA_HOME/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list | grep _l9  
mysql -u root -p -e "USE bigdata_project; SELECT COUNT(*) FROM city_sales_rt; SELECT COUNT(*) FROM order_timeout_alerts;"
```

基线期望：两张 MySQL 表都是 0 行，4 个 Topic 都存在，没有你忘记停止的旧 Job。

6. 故障实验 A：TaskManager 故障 - 验证 State + Timer + Offset 恢复

这是本讲最重要的实验。我们使用订单超时 Job，因为它有最典型的 Keyed State 和 Event-Time Timer。目标不是“杀一个进程看看会不会回来”，而是证明：故障前保存的活动订单状态和定时器，在恢复后仍然存在。

TaskManager 故障：Checkpoint 如何把“状态 + 输入位置”一起恢复



关键：恢复的不只是“内存值”，还包括 Kafka Source 的消费位置；因此重放发生，但最终状态可保持一致。

演示数据分成两阶段。第一阶段只发 CREATE，让 O5001、O5002、O5999 都形成活动 State，然后等待至少一个 Completed Checkpoint。此时再杀 TaskManager。恢复后第二阶段再发 PAY 和推进 Event Time。

订单	阶段 1	阶段 2	最终期望
O5001	CREATE 10:00	PAY 10:05	不超时
O5002	CREATE 10:01	无 PAY	10:16 产生唯一 TIMEOUT_ALERT
O5999	CREATE 10:03	PAY 10:06	不超时
O5998	无	CREATE/PAY 10:20 左右	只负责把 Watermark 推进，不应超时

7. TaskManager 故障实验：保姆式操作步骤

1. 启动订单超时 Job：`./10\_run\_timeout.sh`。
2. 打开 Flink Web UI (<http://<VM-IP>:8081>)，确认 Job = RUNNING。
3. 发送阶段 1：`./10\_send\_timeout\_phase1.sh`。
4. 进入 Job → Checkpoints，必须等到至少一个 Checkpoint 状态为 COMPLETED。记下 Checkpoint ID。
5. 先 Dry Run：`./10\_kill\_taskmanager.sh`，确认它找到的是 TaskManager PID。
6. 真正终止：`CONFIRM\_KILL=YES ./10\_kill\_taskmanager.sh`。
7. 观察 Web UI：可能看到 Task/Subtask unavailable、Job RESTARTING/FAILING 或等待资源。
8. 重新启动 TaskManager：`./10\_restart\_taskmanager.sh`。
9. 等待 Job 回到 RUNNING，并再次出现新的 Completed Checkpoint。
10. 发送阶段 2：`./10\_send\_timeout\_phase2.sh`。
11. 用 `./10\_consume\_alerts.sh` 验 Kafka committed 告警；再查 MySQL。

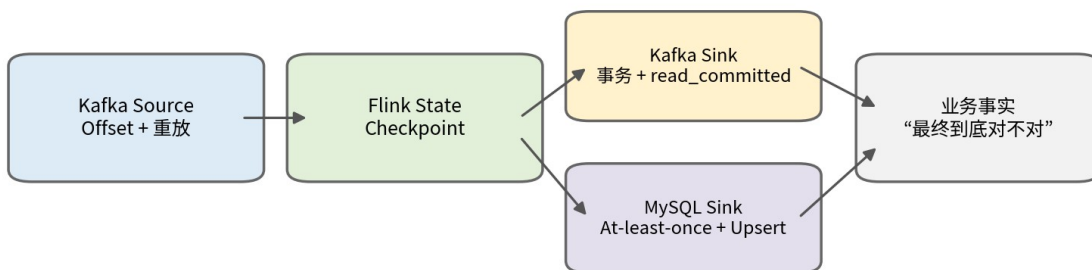
```
mysql -u root -p < sql/10_final_verify_mysql.sql
```

标准答案：这组故障数据最终只允许出现 O5002 一条 TIMEOUT_ALERT (action_time=10:16:00)。O5001、O5999 已在恢复后 PAY，不能误报超时。

8. 底层原理：为什么重启后 Timer 还记得 O5002?

Checkpoint 不是只备份 ValueState。对于一个可恢复的流 Job，它还需要把 Source 消费位置纳入一致性边界。

端到端一致性：不同层次负责不同问题



Checkpoint 能保证 Flink managed state 一致恢复，但“外部世界 exactly-once”仍取决于 Sink 能力和业务幂等设计。

你可以把恢复理解成四步：

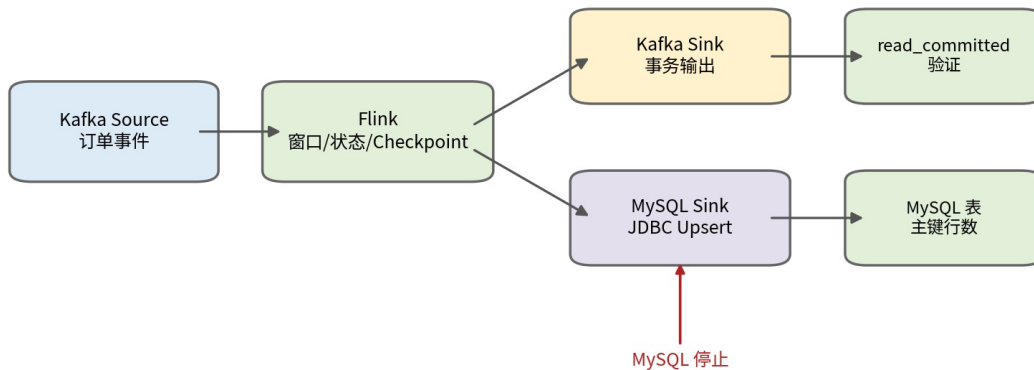
1. Flink 读取最近一次成功 Checkpoint 中的 Keyed State，例如 O5002 的 create_time 和 timerTimestamp。
2. Event-Time Timer 也是受 Flink 管理的状态的一部分，会随 Checkpoint 恢复。
3. Kafka Source 从与该 Checkpoint 一致的位置继续读取，必要时会重放 Checkpoint 之后但尚未被新 Checkpoint 固化的数据。
4. 重放允许发生，但 managed state 的最终效果按 Checkpoint 语义保持一致。

关键概念：“恢复后有重放”不等于“Flink 失去 Exactly-once”。Exactly-once 关注的是状态/结果的最终可观察效果，不是 CPU 指令绝不再次执行。

9. 故障实验 B：MySQL Sink 暂时不可用

第二个实验用城市销售 Job。目的不是证明 MySQL 永远不坏，而是理解当外部 Sink 不可用时，Flink Job、Checkpoint、Kafka 事务输出和 JDBC Upsert 会怎样相互影响。

MySQL Sink 故障：为什么要同时观察 Kafka 与数据库



预期现象不是固定某一个 Job 状态；可能 FAILING / RESTARTING / 重试阻塞。关键是恢复 MySQL 后，最终 Kafka committed 结果与 MySQL 业务表都正确。

1. 先重新清空 Topic 与 city_sales_rt 表，再启动 `./10\_run\_city\_sales.sh`。
2. 等待 Web UI 至少出现一个 Completed Checkpoint。
3. Dry Run：`./10\_mysql\_down.sh`。确认脚本只提示，不会直接停服务。
4. 真正停 MySQL：`CONFIRM\_MYSQL\_STOP=YES ./10\_mysql\_down.sh`。
5. 发送销售演示数据：`./10\_send\_city\_sales\_demo.sh`。
6. 观察 Job 状态和 TaskManager 日志。由于 JDBC 重试/超时节奏不同，Job 可能表现为重试、FAILED/RESTARTING 或一段时间阻塞，不要求机械地出现同一种界面。
7. 恢复 MySQL：`./10\_mysql\_up.sh`。
8. 等待 Job 恢复并出现 Completed Checkpoint。
9. 分别验证 Kafka read_committed 与 MySQL 表。

验收重点：不要只验证 MySQL。Kafka 事务 Sink 与 JDBC Sink 是两条不同边界。恢复后两边的最终业务结果都应与标准答案一致。

10. MySQL 故障实验为什么很重要：两种 Sink 的保证并不相同

链路	机制	故障重试时的关键点
Flink managed state	Checkpoint EXACTLY_ONCE	State + Source 位置一致恢复
Kafka Sink	Kafka transaction + Checkpoint	read_committed 只看已提交事务；未完成事务不会作为 committed 结果暴露
MySQL JdbcSink	at-least-once	失败恢复可能再次执行同一 SQL
MySQL 最终表	PRIMARY KEY + UPSERT	同一业务 Key 重写同一行，保持最终表幂等

因此，本项目中 MySQL “没有重复逻辑行”的原因不是普通 JdbcSink 自动变成事务 exactly-once，而是：即使 SQL 可能重放，稳定业务主键 + Upsert 让同一业务事实最终覆盖同一行。

官方语义：Flink 当前 JDBC 文档仍明确说明 JdbcSink.sink 是 at-least-once；可以通过幂等/Upsert 获得最终效果上的去重，也可以进一步使用基于 XA 的 exactlyOnceSink，但复杂度更高。

11. 可选实验 C：Savepoint - 主动停止与升级式恢复

Checkpoint 主要为故障自动恢复服务；Savepoint 更适合人为触发的停止、升级、迁移。这个实验是可选项，但非常值得做一次。

先让订单超时 Job 产生活态 State，然后在另一个终端执行：

```
# 先找 JobId
$FLINK_HOME/bin/flink list -r

# 触发 Savepoint (目录请确保可写)
$FLINK_HOME/bin/flink savepoint <JobId> hdfs:///bigdata-capstone/savepoints

# 记下输出的 savepoint 路径
# 例如: hdfs:///bigdata-capstone/savepoints/savepoint-xxxx

# 再停止旧 Job
$FLINK_HOME/bin/flink cancel <JobId>
```

随后使用同一个 JAR 从 Savepoint 启动：

```
$FLINK_HOME/bin/flink run -d \
-s hdfs:///bigdata-capstone/savepoints/savepoint-xxxx \
-c edu.project.realtime.OrderTimeoutExternalSinkJob \
target/flink-external-sinks-1.0-SNAPSHOT-all.jar
```

区别：自动 Checkpoint 恢复是“系统坏了自动回到最近一致点”；Savepoint 是“人主动生成可迁移状态快照，再从它启动新 Job”。

12. 可选实验 D：Kafka 短暂停止 - 不要把它和 Producer 停止混为一谈

Producer 暂停只是“暂时没有新数据”，通常不是故障；Kafka Broker 真正停止才是基础设施故障。单节点教学 Kafka 可以做一次短暂停机实验，但风险比前两项高，因此本讲只作为可选。

建议步骤：先确保有 Completed Checkpoint，记录 Job 状态；短暂停 Kafka Broker；观察 Source/Sink 错误与重试；尽快恢复 Broker；观察 Job 是否重新正常并能继续消费。

不要死记界面：不同 Kafka/Flink 版本、网络超时和 restart strategy 会导致 Job 状态转换节奏不同。真正应该比较的是“故障前后 Offset/Checkpoint/最终结果是否一致”，不是要求每个人都看到完全相同的 3 秒钟界面。

13. 离线链最终验收：HDFS + MapReduce + Spark

故障实验做完后，不要忘记整个综合项目还有离线链。最终验收时需要重新证明离线结果仍可重算。


```
# HDFS 原始数据
hdfs dfs -ls -R /bigdata-capstone/raw

# MapReduce 第3讲结果
hdfs dfs -cat /bigdata-capstone/output/mapreduce/city_paid_sales/part-r-00000 | sort

# Spark 第4/5讲 Parquet 结果目录
hdfs dfs -ls -R /bigdata-capstone/output/spark
```

MapReduce 与 Spark 对同一业务问题的标准答案必须一致：

Beijing	6874
Guangzhou	676
Shanghai	6984
Shenzhen	299
TOTAL	14833

为什么还要再验一遍：最终项目验收关注“整个系统链条”。实时系统跑得很好，也不能替代离线历史可重算能力。

14. 实时链最终验收：Kafka + Flink + MySQL

实时链要同时验三个层次：Source 可消费、Flink 可恢复、Sink 最终结果正确。

最终验收：不是一个绿色勾，而是多层证据矩阵

存储	HDFS 输入存在；目录/文件/Block 可验证
离线 MapReduce	城市 PAID 销售额 TOTAL = 14833
离线 Spark	同业务结果 = 14833；Parquet 可回读
Kafka	Topic/Partition/Offset/Consumer Group 可观察
Flink 状态	Checkpoint 成功；TaskManager 故障后状态/Timer 可恢复
Kafka Sink	read_committed 看到预期 committed 结果
MySQL Sink	主键行数正确；重复重放不产生额外逻辑行
业务验收	城市窗口3行；超时告警严格符合标准答案

城市销售标准答案：

```
[10:01,10:02) Beijing count=1 sales=258.00
[10:01,10:02) Shanghai count=1 sales=499.00
[10:02,10:03) Beijing count=1 sales=118.00
MySQL city_sales_rt rows = 3
```

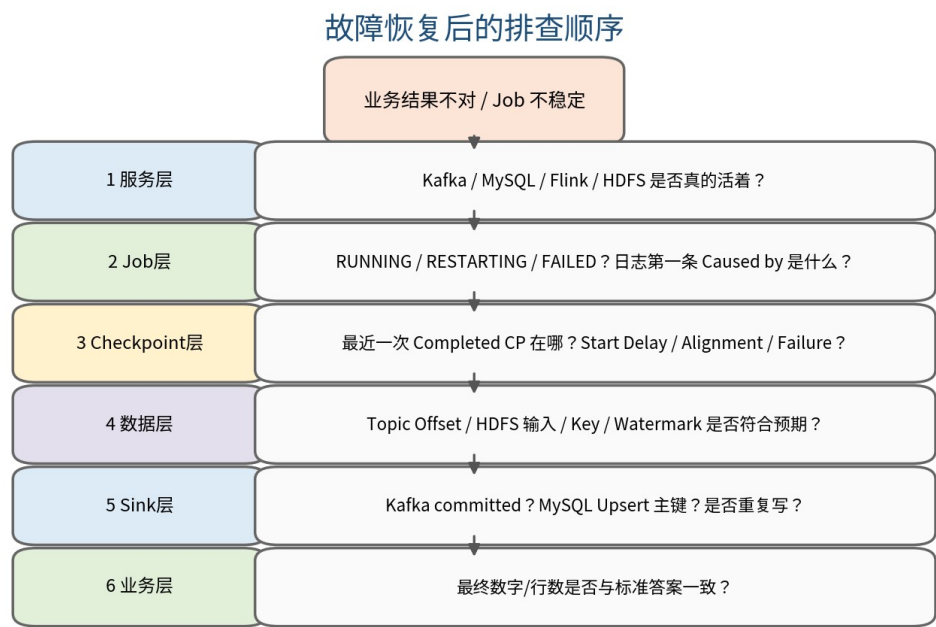
第 9 讲超时告警标准答案：

```
03002 TIMEOUT_ALERT 10:16:00
03004 TIMEOUT_ALERT 10:18:00
MySQL order_timeout_alerts rows = 2
```

第 10 讲 TaskManager 故障恢复专用标准答案：

```
05002 TIMEOUT_ALERT 10:16:00
05001 no timeout
05999 no timeout
```

15. 最终排错树：以后遇到问题固定按这个顺序



这个顺序的核心思想是：先确认基础设施，再看 Job，再看 Checkpoint，再看数据流，再看 Sink，最后才下业务结论。不要把所有问题都归咎于“内存不够”或“Kafka 有问题”。

现象	第一检查项	不要先做什么
Flink Job 不 RUNNING	Web UI / TaskManager 日志 / 第一条 Caused by	不要先调大内存
Checkpoint 一直失败	Start delay / alignment / Sink / State	不要直接关闭 Checkpoint
Kafka 有数据但 Flink 没输出	Group/Offset/Watermark/Job Status	不要先重建 Topic
Kafka committed 没结果	Completed Checkpoint / transaction / read_committed	不要改成 read_uncommitted 逃避问题
MySQL 行数多了	PRIMARY KEY / UPSERT / Source 是否重复	不要只 DELETE 表重跑
金额不等标准答案	业务过滤/Join/重复事件/窗口边界	不要只看 Job=SUCCESS

16. 综合项目最终能力清单：做到这些才算真正闭环

- 1. 我能区分 Linux 本地路径与 HDFS 路径，并能验证 HDFS Block/副本。
- 2. 我能解释并运行一次完整 MapReduce Job，知道 Mapper、Shuffle、Reducer 在哪里发生。
- 3. 我能用 Spark DataFrame 读取 HDFS、多表 Join、聚合、写 Parquet，并能看 explain / Stage / Task。
- 4. 我能独立创建 Kafka Topic，理解 Partition、Offset、Consumer Group、Lag。
- 5. 我能让 Flink 从 Kafka 读取 JSON，使用 Event Time、Watermark 和 Window。
- 6. 我能用 Keyed State + Event-Time Timer 实现订单超时状态机。
- 7. 我能启用 Checkpoint，解释 State + Source Offset 如何共同恢复。
- 8. 我能解释 Kafka Sink exactly-once 与 MySQL Upsert 幂等为什么不是同一种机制。

9. 我能故意杀 TaskManager，并用证据证明恢复后的 State/Timer 仍正确。
10. 我能在 MySQL 故障后恢复服务，并再次验证 Kafka committed 与 MySQL 最终表。
11. 我能按“服务 → Job → Checkpoint → 数据 → Sink → 业务”顺序排错。
12. 我能拿一个新需求，判断哪些数据走 HDFS/Spark，哪些事件走 Kafka/Flink，而不是机械堆技术。

最低闭环：如果上面 12 项你都能在没有现成答案时独立完成，那么这套“大数据处理技术”已经从知识点集合变成了一个真正可操作的系统心智模型。

17. 练习题

1. 为什么 TaskManager 故障后即使 Kafka Source 重放部分消息，也不等于 Flink managed state 一定重复计算？
2. 为什么 MySQL JdbcSink 使用 Upsert 后“最终表不重复”，仍不能简单宣称整个 JDBC 链路是事务 exactly-once？
3. 如果 Job 恢复为 RUNNING，但 O5001 误报了 TIMEOUT_ALERT，你认为应该优先检查哪三类证据？
4. 为什么做故障实验时必须先等一个 Completed Checkpoint？
5. Checkpoint 与 Savepoint 的使用目的分别是什么？
6. 如果 Kafka read_committed 没输出，但 MySQL 有结果，你会怎么排查？

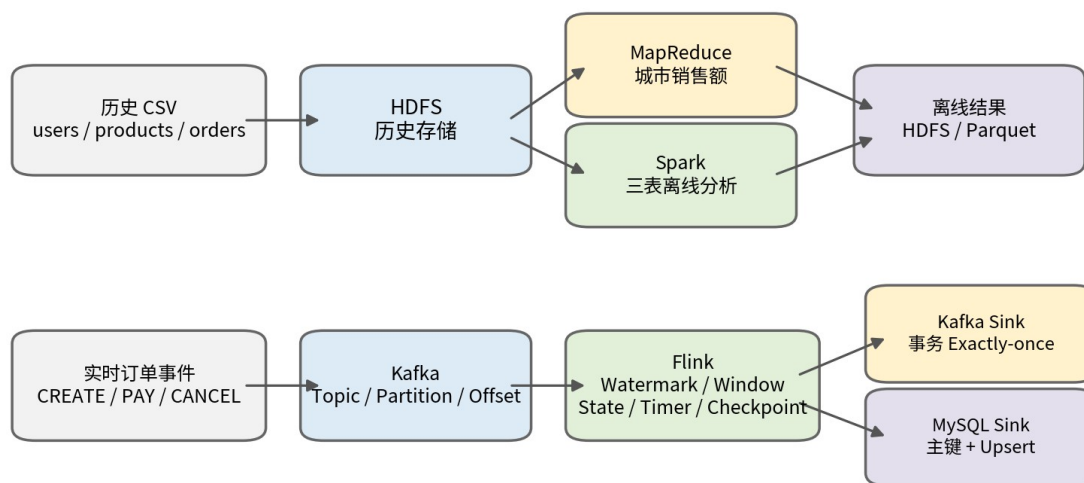
参考答案（先自己完成再看）

1. Checkpoint 会一致保存 Flink managed state 和 Source 消费位置；重放允许发生，但恢复从一致点继续，最终状态语义由 Checkpoint 保证。
2. 普通 JdbcSink 是 at-least-once。Upsert 依靠稳定业务主键把重复 SQL 变成同一行覆盖，是业务幂等，不等于 XA 事务 exactly-once。
3. 先看恢复使用的 Checkpoint、O5001 对应 PAY 是否被 Source 正确消费、Watermark/Timer 是否按预期推进，再看代码/日志。
4. 没有成功 Checkpoint，故障后就缺少可恢复的一致状态点；实验无法证明“从已保存 State 恢复”。
5. Checkpoint 主要用于系统自动容错恢复；Savepoint 更适合人为触发的停机、升级、迁移和回滚。
6. 先确认 Kafka Sink checkpoint/transaction 是否完成，再确认消费者 isolation.level=read_committed、Topic 名称和 Offset；不要因为 MySQL 有结果就假设 Kafka 一定也提交了事务。

18. 项目收束：从“学框架”到“理解数据系统”

十讲综合项目到这里结束。你已经沿同一份业务数据分别走过离线链和实时链，并且不只做了正常路径，还验证了故障路径。

综合项目最终系统：离线链 + 实时链



最终验收不是“某个 Job SUCCESS”，而是两条数据链的业务结果与故障恢复都可证明。

现在再看这些技术，不应该是六门互不相干的软件：HDFS 负责可靠存储，YARN/Flink 资源层负责计算资源，MapReduce/Spark 负责历史批处理，Kafka 负责事件日志与解耦，Flink 负责有状态实时计算，MySQL/Kafka Sink 负责把结果交付给下游。

下一阶段建议：不要马上继续堆更多框架。更有价值的是把这套综合项目真正手工跑通一次，再让学生以不同业务需求做变体：例如实时用户 30 分钟消费风控、异常登录告警、实时商品 Top-N 等。

官方核对：Apache Flink Task Failure Recovery / Kafka Connector Delivery Guarantees / JDBC Connector。